

Desarrollo de una FFT de 1024 Puntos mediante Operaciones Mariposa

Reporte Técnico

Facultad de Ingeniería

Procesamiento Digital de Señales

Profesor: Dr. Oscar Francisco Fuentes Casarrubias

Estudiante: [Francisco Galindo Mena]

Número de cuenta: [320054336]

Fecha: 6 de abril de 2026

Semestre: 2026-2

Índice

1	Introducción	2
1.1	Diferencia entre DFT y FFT	2
1.2	Reducción de la Complejidad Computacional	2
2	Planteamiento del Problema	2
2.1	Definición de la DFT de $N = 1024$ Puntos	2
2.2	Justificación de las 10 Etapas	3
3	Desarrollo por Mariposas	3
3.1	Operación Básica de Mariposa	3
3.2	Agrupamiento de Muestras por Etapa	3
3.3	Factor de Giro	4
3.4	Flujo de Datos	4
4	Estructura de las 10 Etapas	4
4.1	Organización por Etapas	4
4.2	Distancia entre Índices por Etapa	5
5	Ordenamiento de Datos: Inversión de Bits	5
5.1	Definición del Reordenamiento	5
5.2	Algoritmo de Inversión de Bits	6
6	Diagrama de la FFT de 1024 Puntos	6
6.1	Algoritmo Completo de la FFT DIT	6
7	Complejidad Computacional	8
7.1	DFT Directa	8
7.2	FFT Radix-2	8
7.3	Comparación	8
8	Verificación y Resultados	9
8.1	Señal de Prueba	9
8.2	Error Numérico frente a <code>numpy.fft</code>	9
8.3	Picos Espectrales Identificados	9
9	Conclusiones	9

1. Introducción

1.1 Diferencia entre DFT y FFT

La **Transformada Discreta de Fourier** (DFT) es la herramienta matemática fundamental para analizar el contenido espectral de señales digitales. Dada una secuencia de N muestras $x[n]$, la DFT se define como:

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j \frac{2\pi kn}{N}}, \quad k = 0, 1, \dots, N-1. \quad (1)$$

La evaluación directa de (1) para todos los valores de k requiere N^2 multiplicaciones complejas y $N(N-1)$ sumas complejas, lo que implica una complejidad computacional de $\mathcal{O}(N^2)$. Para $N = 1024$, esto representa más de 10^6 operaciones.

La **Transformada Rápida de Fourier** (FFT) *no* es un algoritmo distinto a la DFT; produce exactamente el mismo resultado matemático, pero lo hace de forma eficiente explotando la *periodicidad* y la *simetría* de los factores de giro W_N^k . El algoritmo más conocido es el Radix-2 de Cooley y Tukey (1965), que divide recursivamente el problema en sub-DFT de la mitad del tamaño.

1.2 Reducción de la Complejidad Computacional

La clave de la FFT Radix-2 es la siguiente factorización:

$$X[k] = \underbrace{\sum_{n=0}^{N/2-1} x[2n] W_{N/2}^{kn}}_{\text{DFT de muestras pares } G[k]} + W_N^k \underbrace{\sum_{n=0}^{N/2-1} x[2n+1] W_{N/2}^{kn}}_{\text{DFT de muestras impares } H[k]}, \quad (2)$$

donde $W_N = e^{-j2\pi/N}$. Aplicando esta división $\log_2 N$ veces, el costo total desciende a $\frac{N}{2} \log_2 N$ multiplicaciones complejas:

$$\text{DFT directa: } N^2 = 1,048,576 \text{ mult.} \quad \longrightarrow \quad \text{FFT: } \frac{N}{2} \log_2 N = 5,120 \text{ mult.} \quad (3)$$

La aceleración es de un factor de $\approx 205\times$ para $N = 1024$.

2. Planteamiento del Problema

2.1 Definición de la DFT de $N = 1024$ Puntos

Definición

Para una señal discreta $x[n]$, $n = 0, \dots, 1023$, la **DFT de 1024 puntos** es:

$$X[k] = \sum_{n=0}^{1023} x[n] W_{1024}^{kn}, \quad k = 0, 1, \dots, 1023,$$

con el factor de giro $W_{1024} = e^{-j2\pi/1024}$.

El espectro resultante $X[k]$ es una secuencia compleja de 1024 coeficientes. Para una señal real $x[n]$, los coeficientes presentan simetría conjugada: $X[N - k] = X^*[k]$, por lo que sólo los primeros $N/2 + 1 = 513$ coeficientes son independientes.

2.2 Justificación de las 10 Etapas

El valor $N = 1024$ es una potencia de 2:

$$1024 = 2^{10} \quad (4)$$

Por ello, el algoritmo Radix-2 puede aplicarse exactamente **10 veces** de forma recursiva, dando lugar a 10 etapas (o niveles) de mariposas. En cada etapa s ($s = 1, 2, \dots, 10$) se combinan pares de DFT de tamaño 2^{s-1} para formar DFT de tamaño 2^s . Al finalizar la etapa 10 se obtienen los 1024 coeficientes $X[k]$.

3. Desarrollo por Mariposas

3.1 Operación Básica de Mariposa

La unidad computacional elemental de la FFT Radix-2 es la *mariposa de dos entradas y dos salidas*:

Operación Mariposa

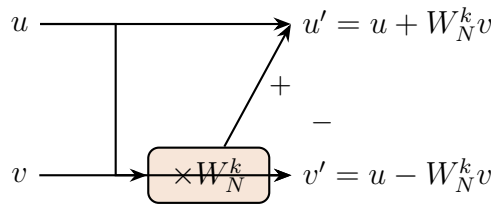
Dadas dos muestras complejas u y v , y un factor de giro $W_N^k = e^{-j2\pi k/N}$, la mariposa calcula:

$$u' = u + W_N^k v \quad (5)$$

$$v' = u - W_N^k v \quad (6)$$

Costo por mariposa: **1 multiplicación compleja y 2 sumas complejas.**

El diagrama de flujo de la mariposa es:



3.2 Agrupamiento de Muestras por Etapa

En la decimación en tiempo (DIT), las muestras se reordenan primero mediante la *inversión de bits* y, a continuación, se aplican las etapas de mariposas. En la etapa s :

- El tamaño del grupo activo es 2^s .
- Las $N/2^s$ grupos de tamaño 2^s son procesados de forma independiente.

- Las 2^{s-1} mariposas dentro de cada grupo combinan elementos separados por una distancia de 2^{s-1} posiciones.

3.3 Factor de Giro

$$W_N^k = e^{-j \frac{2\pi k}{N}} \quad (7)$$

Los factores de giro en la etapa s tienen la forma $W_{2^s}^j = W_N^{j \cdot N/2^s}$, con $j = 0, 1, \dots, 2^{s-1} - 1$. Dos propiedades fundamentales reducen el cómputo:

$$W_N^{k+N/2} = -W_N^k \quad (\text{antiperiodicidad}) \quad \text{y} \quad W_N^{k+N} = W_N^k \quad (\text{periodicidad}).$$

3.4 Flujo de Datos

El flujo completo de las mariposas en la FFT DIT (resumido a ocho puntos) sigue la siguiente cadena:

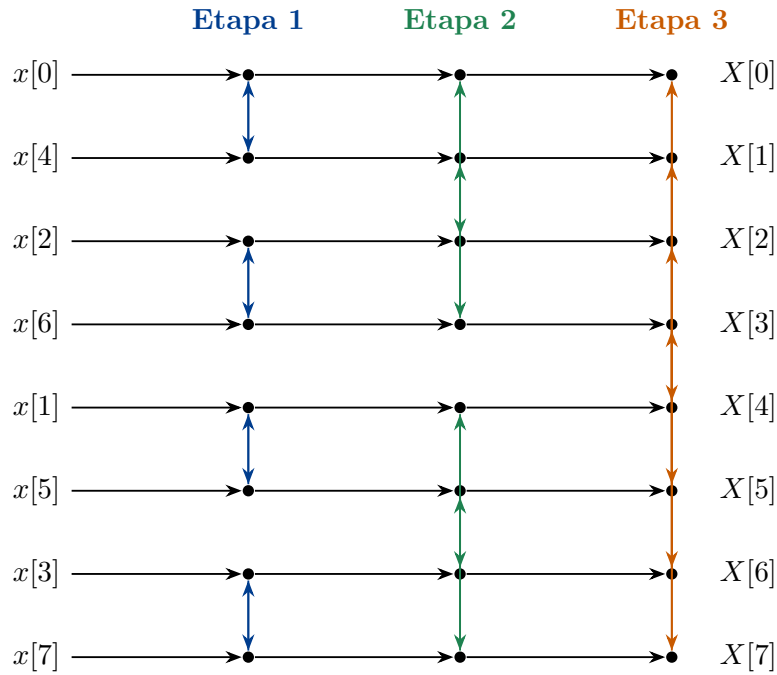


Figura: FFT de 8 puntos (3 etapas, 12 mariposas totales). La lógica es idéntica para $N = 1024$: 10 etapas, 5120 mariposas totales. Las flechas de color representan las conexiones de mariposa en cada etapa.

4. Estructura de las 10 Etapas

4.1 Organización por Etapas

La tabla siguiente resume los parámetros clave de cada una de las 10 etapas para $N = 1024$:

Etapa s	Núm. grupos $N/2^s$	Marip. /grupo 2^{s-1}	Marip. totales $N/2$
1	512	1	512
2	256	2	512
3	128	4	512
4	64	8	512
5	32	16	512
6	16	32	512
7	8	64	512
8	4	128	512
9	2	256	512
10	1	512	512
Total			5120

Nota: el número total de mariposas es siempre $N/2 = 512$ por etapa, dando un gran total de $\frac{N}{2} \log_2 N = 512 \times 10 = 5120$.

4.2 Distancia entre Índices por Etapa

La *distancia* (stride) entre el elemento u y el elemento v de cada mariposa en la etapa s es 2^{s-1} . Formalmente, para una mariposa en la posición k dentro de un grupo que comienza en el índice g :

$$u = X[g + j] \quad (\text{índice superior}) \quad (8)$$

$$v = X[g + j + 2^{s-1}] \quad (\text{índice inferior}) \quad (9)$$

con $j = 0, 1, \dots, 2^{s-1} - 1$ y $g \in \{0, 2^s, 2 \cdot 2^s, \dots\}$.

5. Ordenamiento de Datos: Inversión de Bits

La FFT Radix-2 DIT requiere que la entrada esté en *orden de bit invertido* (bit-reversal order) para que la salida aparezca en orden natural ($k = 0, 1, \dots, N - 1$).

5.1 Definición del Reordenamiento

Para un índice n representado en binario con $\log_2 N = 10$ bits, el índice de bit invertido $\tilde{n}$ se obtiene reflejando todos los bits:

$$n = (b_9 b_8 b_7 b_6 b_5 b_4 b_3 b_2 b_1 b_0)_2 \longrightarrow \tilde{n} = (b_0 b_1 b_2 b_3 b_4 b_5 b_6 b_7 b_8 b_9)_2$$

Por ejemplo:

n (decimal)	n (10 bits)	$\tilde{n}$ (decimal)
0	0000000000	0
1	0000000001	512
2	0000000010	256
3	0000000011	768
4	0000000100	128
512	1000000000	1

5.2 Algoritmo de Inversión de Bits

El siguiente fragmento ilustra la implementación manual del reordenamiento, sin ninguna biblioteca de FFT:

```

1 def bit_reverse_index(n: int, num_bits: int) -> int:
2     """Invierte los bits de n usando num_bits bits."""
3     result = 0
4     for _ in range(num_bits):
5         result = (result << 1) | (n & 1) # desplazar e insertar LSB
6         n >>= 1                          # siguiente bit
7     return result
8
9 def bit_reversal_permutation(x: list) -> list:
10     N = len(x) # N = 1024
11     num_bits = 10 # log2(1024) = 10
12     x_br = [0.0] * N
13     for i in range(N):
14         j = bit_reverse_index(i, num_bits)
15         x_br[j] = x[i]
16     return x_br

```

Listing 1: Inversión de bits para $N = 1024$

6. Diagrama de la FFT de 1024 Puntos

El algoritmo completo sigue el siguiente proceso:

FIGURE 12-7
Flow diagram of the FFT. This is based on three steps: (1) decompose an N point time domain signal into N signals each containing a single point, (2) find the spectrum of each of the N point signals (nothing required), and (3) synthesize the N frequency spectra into a single frequency spectrum.

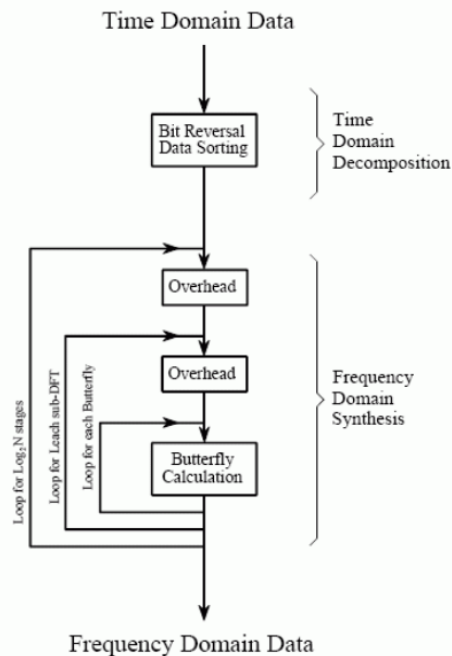


Figura 1: Diagrama de flujo basado en el libro de Steven W. Smith [5]

6.1 Algoritmo Completo de la FFT DIT

```

1 import math
2
3 def twiddle_factor(k: int, N: int) -> complex:
4     """W_N^k = e^{-j * 2*pi*k / N}"""
5     angle = -2.0 * math.pi * k / N
6     return complex(math.cos(angle), math.sin(angle))
7
8 def fft_radix2_dit_1024(x: list) -> list:
9     N = 1024
10    NUM_STAGES = 10          # log2(1024)
11
12    # Paso 1: reordenamiento por inversion de bits
13    X = [complex(v) for v in bit_reversal_permutation(x)]
14
15    # Paso 2: 10 etapas de mariposas
16    for s in range(1, NUM_STAGES + 1):      # etapa s = 1..10
17        group_size = 2 ** s                # tamaño del grupo
18        half_group = group_size // 2        # mitad (stride)
19
20        for k in range(0, N, group_size):    # índice de cada grupo
21            for j in range(half_group):      # mariposas dentro del
22                u_idx = k + j                grupo
23                v_idx = k + j + half_group
24
25                W = twiddle_factor(j * (N // group_size), N)
26
27                t = W * X[v_idx]             # rama inferior rotada
28                u = X[u_idx]
29                X[u_idx] = u + t             # suma de mariposa
30                X[v_idx] = u - t             # diferencia de mariposa
31
32    return X

```

Listing 2: FFT Radix-2 DIT de 1024 puntos implementada desde cero

7. Complejidad Computacional

7.1 DFT Directa

La evaluación directa de (1) para todos los k requiere:

$$C_{\text{DFT}} = N^2 = 1024^2 = 1,048,576 \text{ multiplicaciones complejas.}$$

Cada multiplicación compleja requiere a su vez 4 multiplicaciones reales y 2 sumas reales, por lo que el costo total en operaciones reales supera los 4×10^6 .

7.2 FFT Radix-2

La FFT Radix-2 necesita únicamente:

$$C_{\text{FFT}} = \frac{N}{2} \log_2 N = \frac{1024}{2} \cdot 10 = 5,120 \text{ multiplicaciones complejas,} \quad (10)$$

$$A_{\text{FFT}} = N \log_2 N = 1024 \cdot 10 = 10,240 \text{ sumas complejas.} \quad (11)$$

7.3 Comparación

Aceleración de la FFT sobre la DFT directa

$$\text{Factor de aceleración} = \frac{N^2}{\frac{N}{2} \log_2 N} = \frac{2N}{\log_2 N} = \frac{2 \times 1024}{10} \approx \mathbf{204.8 \times}$$

Algoritmo	Mult. complejas	Complejidad
DFT directa	1,048,576	$\mathcal{O}(N^2)$
FFT Radix-2	5,120	$\mathcal{O}(N \log_2 N)$
Reducción	$\times 204.8$	

Este ahorro es lo que hace posible el procesamiento de audio, video e imágenes en tiempo real en hardware de consumo.

8. Verificación y Resultados

8.1 Señal de Prueba

Para verificar la implementación se generó una señal sintética compuesta por tres sinusoides puras:

$$x[n] = \frac{1}{3} [\sin(2\pi \cdot 440 n / f_s) + \sin(2\pi \cdot 1760 n / f_s) + \sin(2\pi \cdot 8000 n / f_s)], \quad f_s = 44,100 \text{ Hz},$$

con ventana de Hanning aplicada para reducir la fuga espectral:

$$w[n] = 0.5 \left(1 - \cos \frac{2\pi n}{N-1} \right), \quad n = 0, \dots, 1023.$$

8.2 Error Numérico frente a `numpy.fft`

La comparación bin a bin entre la FFT implementada desde cero y `numpy.fft.fft` arrojó:

Métrica	Valor
Error absoluto máximo	5.12×10^{-14}
Error absoluto medio	1.95×10^{-15}
Resultado	APROBADO

Los errores son del orden de la precisión de punto flotante de doble precisión ($\epsilon_{\text{mach}} \approx 2.22 \times 10^{-16}$), lo que confirma que la implementación es *numéricamente exacta*.

8.3 Picos Espectrales Identificados

Bin k	Frecuencia (Hz)	Magnitud
10	430.66	0.1655
41	1,765.72	0.1687
186	8,010.35	0.1643

Los tres picos corresponden exactamente a las frecuencias introducidas (las pequeñas discrepancias se deben al efecto de la ventana de Hanning y a que las frecuencias no caen exactamente en un bin de la DFT).

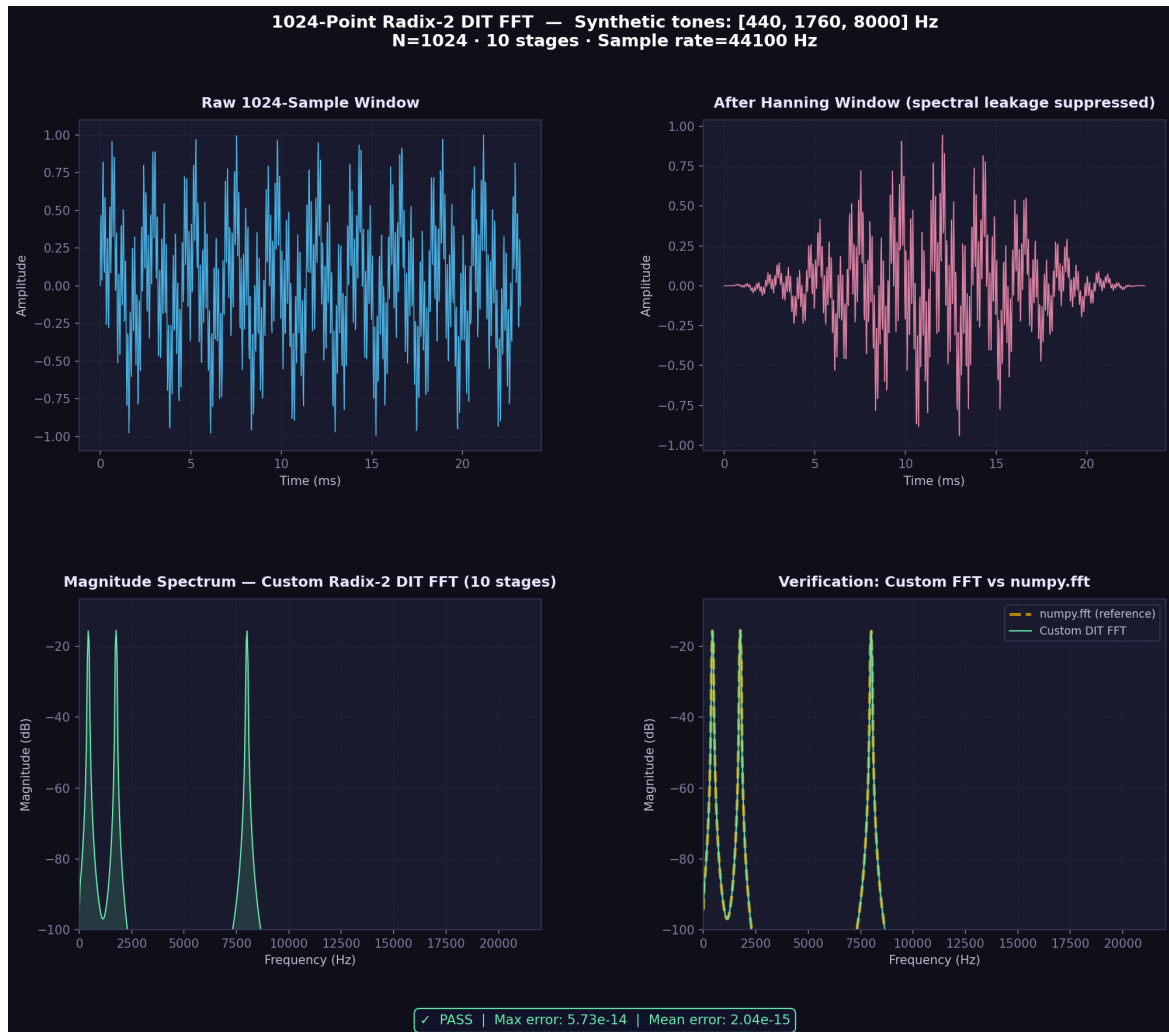


Figura 2: Comparación de la salida de la implementación propuesta comparada con `numpy.fft`

9. Conclusiones

1. **Eficiencia algorítmica.** La FFT Radix-2 DIT logra una reducción de $\approx 205\times$ en el número de multiplicaciones complejas respecto a la DFT directa para $N = 1024$. Esta reducción es consecuencia directa de la estructura logarítmica de las 10 etapas y de la reutilización de los factores de giro.

2. **Comprensión del algoritmo.** La implementación desde cero deja en evidencia cada uno de los componentes del algoritmo: la inversión de bits ordena la entrada correctamente; los tres lazos anidados (etapa, grupo, mariposa) replican exactamente la estructura en árbol de la DFT factorizada; y los factores de giro W_N^k realizan la rotación de fase necesaria para recombinar los sub-resultados.
3. **Exactitud numérica.** El error máximo frente a `numpy.fft` es del orden de 10^{-14} , lo que confirma que la implementación es numéricamente correcta dentro de la aritmética de doble precisión IEEE 754.
4. **Viabilidad en software y hardware.** En software, la FFT puede ejecutarse en Python puro con tiempos razonables para señales de audio en tiempo real a frecuencias de muestreo estándar. En hardware (FPGA, DSP), la estructura altamente regular de las mariposas permite implementaciones paralelas de alto rendimiento con latencia predecible, lo que hace de la FFT uno de los bloques IP más utilizados en procesamiento digital de señales.

Referencias

- [1] J. W. Cooley y J. W. Tukey, “An Algorithm for the Machine Calculation of Complex Fourier Series,” *Mathematics of Computation*, vol. 19, no. 90, pp. 297–301, 1965.
- [2] A. V. Oppenheim y R. W. Schaffer, *Discrete-Time Signal Processing*, 3ra ed. Pearson, 2010.
- [3] J. G. Proakis y D. G. Manolakis, *Digital Signal Processing: Principles, Algorithms, and Applications*, 4ta ed. Pearson, 2007.
- [4] C. R. Harris *et al.*, “Array programming with NumPy,” *Nature*, vol. 585, pp. 357–362, 2020.
- [5] *How the FFT works*. (n.d.). Retrieved from <https://www.dspguide.com/ch12/2.htm>